LP3 Assignment A1

Roll no: 41168

Problem Statement:
Write a non-recursive and recursive program to calculate Fibonacci numbers and analyze their time and space complexity.

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

int rec_fibno(int n){
    if(n == 0)
        return 0;
    if(n == 1)
        return 1;
    return rec_fibno(n - 1) + rec_fibno(n - 2);
}

int itr_fibno(int n){
    int a = 0, b = 1, c;
    if(n == 0)
        return a;
    for(int i = 1; i < n; i++){
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}

int main() {
    int n;
    while(true) {
        cout << "Enter the number which you want to find Fibonacci for (or -1 to exit): ";
        cin >> n;
        if(n == -1)
            break;

        auto start = high_resolution_clock::now();
        int recResult = rec_fibno(n);
        auto end = high_resolution_clock::now();
        long long recTime = duration_cast<nanoseconds>(end - start).count();

        start = high_resolution_clock::now();
        int itrResult = itr_fibno(n);
        end = high_resolution_clock::now();
        long long itrTime = duration_cast<nanoseconds>(end - start).count();
```

```
        cout << "Recursive Fibonacci of " << n << ": " << recResult << "  (Time: " << recTime << "
ns)" << endl;
        cout << "Iterative Fibonacci of " << n << ": " << itrResult << "  (Time: " << itrTime << " ns)"
<< endl;
    }
    return 0;
}
```

Output:

